

Ansible

Was ist Ansible?

Grundlegend ist Ansible ein Open-Source-Tool, das zur Automatisierung der Verwaltung von Systemen dient. Es ermöglicht die Konfiguration und Steuerung von Systemen aus der Ferne und sorgt dafür, dass diese sich in einem gewünschten Zustand befinden. (vgl. [ANS1])

In den meisten Ansible Umgebungen gibt es drei wesentliche Hauptkomponenten:

- **Control Node**
Ein System, auf dem Ansible installiert ist. Auf einem Control Node werden Ansible-Befehle wie `ansible` oder `ansible-inventory` ausgeführt. (vgl. [ANS1])
- **Inventory**
Eine Liste von verwalteten Knoten, die logisch organisiert sind. Das Inventar befindet sich dabei auf dem Control Node und wird hauptsächlich für die Beschreibung und Verwaltung der einzelnen Hosts genutzt, auf die Ansible bzw. der Control Node Zugriff haben sollte. (vgl. [ANS1])
- **Managed Node**
Ein entferntes System oder Host, das von Ansible gesteuert wird. (vgl. [ANS1])

Nachfolgendes Bild visualisiert das Zusammenspiel zwischen den drei Komponenten:

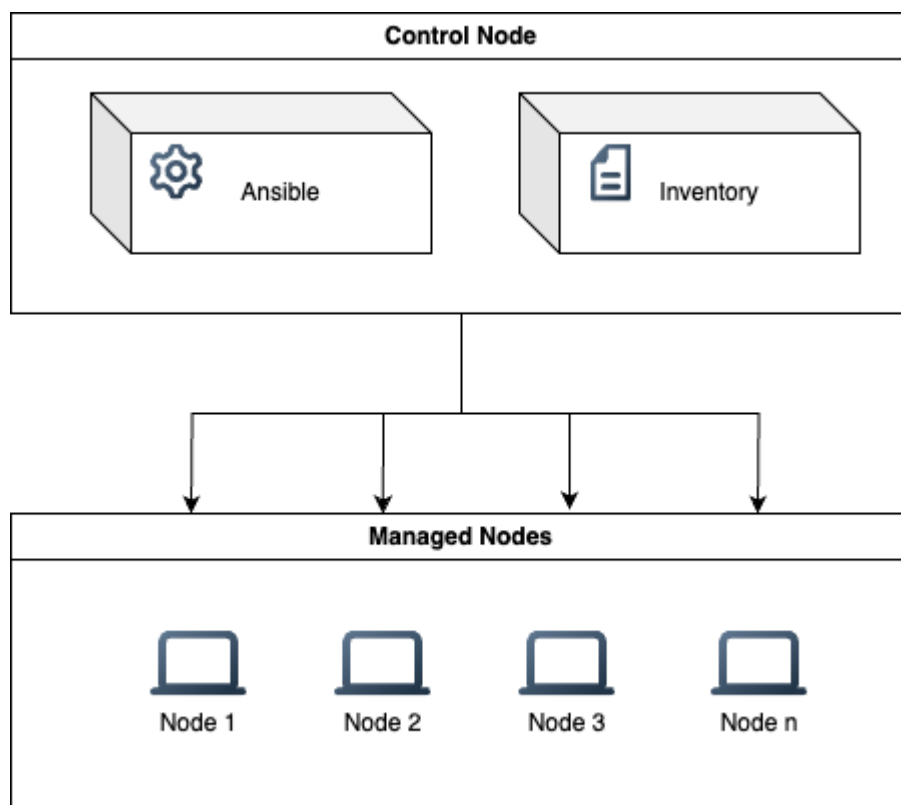


Abbildung 1. Zusammenspiel zwischen Control Node, Inventory und Managed Node

Allgemeine Informationen zu Ansible

Ansible ist ein Open Source Automatisierungstool. Hauptziel ist es, dass die Komplexität der Automatisierung reduziert wird und es überall ausgeführt werden kann. Nahezu jede Aufgabe bzw. Task lässt sich mit Ansible automatisieren. Typische Anwendungsfälle sind: (vgl. [ANS2])

- Eliminierung von Wiederholungen und Vereinfachung von Workflows
- Verwalten und Pflegen von Systemkonfigurationen
- Kontinuierliches Bereitstellen komplexer Software
- Durchführen von Updates ohne Ausfallzeiten

Damit auch diese Vorhaben erreicht werden können, nutzt Ansible die sogenannten **Playbooks**. Ein Playbook ist dabei nichts weiter als ein einfaches, von Menschen lesbares Skript, worin der gewünschte Systemzustand eines Hosts deklariert wird. Ansible stellt hierfür sicher, dass das entsprechende System in diesem Zustand verbleibt. (vgl. [ANS2])

Als Automatisierungstechnologie ist Ansible nach folgenden Prinzipien aufgebaut: (vgl. [ANS2])

- **Agentenlose Architektur**

Über die eigentliche IT Infrastruktur muss keine zusätzliche Software installiert werden. Es gibt lediglich den Control Node mit der Ansible Installation. Dadurch verringert sich natürlich auch der Wartungsaufwand erheblich.

- **Einfachheit**

Playbooks nutzen als Syntax YAML. YAML lässt sich wie eine Dokumentation lesen, wodurch Interpretation und Erstellung sehr vereinfacht wird. Weiterhin ist Ansible dezentralisiert und nutzt SSH um sich mit Systemen zu verbinden, was die Kommunikation und Zugriff als sehr einfach gestaltet.

- **Skalierbarkeit und Flexibilität**

Ansible nutzt ein modulares Designprinzip, wodurch es für die unterschiedlichsten Betriebssysteme, Cloudplattformen oder Netzwerkgeräte Support bietet. Durch dieses Prinzip lassen sich Systeme auch sehr schnell skalieren und entsprechend anpassen.

- **Idempotenz und Vorhersagbarkeit**

Sofern ein System sich im gewünschten Zustand befindet, welcher im Playbook beschrieben ist, wird am Zustand nichts mehr geändert. Das ist auch dann der Fall, wenn ein Playbook mehrmals hintereinander ausgeführt wird.

Erste Automatisierungsschritte

Vorbereitung der Umgebung

Für das komplette Kapitel über Ansible wird eine LXC Containerlandschaft verwendet. Der Aufbau sieht dabei folgendermaßen aus:

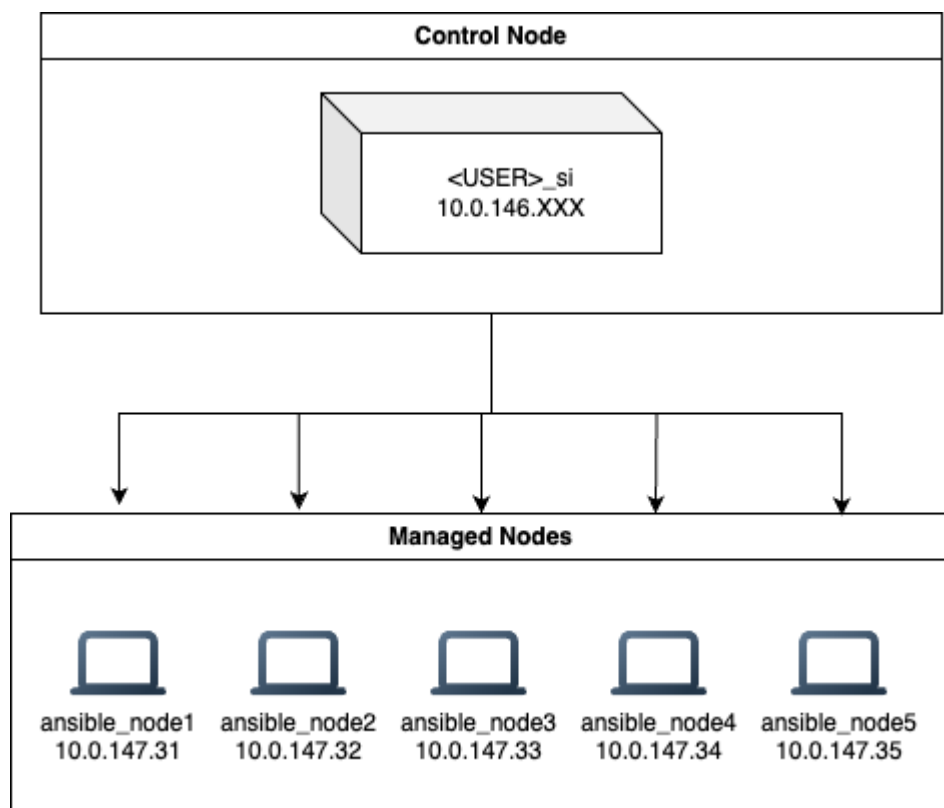


Abbildung 2. Übersicht der Testumgebung

Um eine reibungslose Kommunikation zwischen dem Control Node und den Managed Nodes herstellen zu können, wird SSH verwendet.

Die Managed Nodes selbst sind an die Domäne `b11b.local` angebunden. Dadurch kann über `ssh <USER>@<IP_NODE>` auf den entsprechenden Managed Nodes zugegriffen werden.

Damit eine reibungslose Kommunikation erfolgen kann, sollte im Anschluss auf dem Control Node noch ein entsprechender Benutzer angelegt werden, wie er auch auf den Managed Nodes zu finden ist. Das bedeutet, dass der Benutzer auf dem Control Node den gleichen Namen und das gleiche Passwort haben sollte, wie auf den Managed Nodes.

Installation von Ansible auf dem Control Node

Damit Ansible auf der entsprechenden Control Node installiert werden kann, muss zuerst das entsprechende Paket via `apt` installiert werden:

```
apt update && apt install ansible
```

TERMINAL

Mit Hilfe des Befehls `ansible --version` kann überprüft werden, ob die Installation erfolgreich war.

Sollte hierbei der Fehler `ERROR: Ansible could not initialize the preferred locale: unsupported locale setting` auftreten, so müssen die Locale-Einstellungen angepasst werden:

```
dpkg-reconfigure locales
```

TERMINAL

Im Folgescreen müssen die entsprechenden Locales ausgewählt werden, welche benötigt werden. In der Regel reicht es aus, wenn `en_US.UTF-8` und `de_DE.UTF-8` ausgewählt sind:

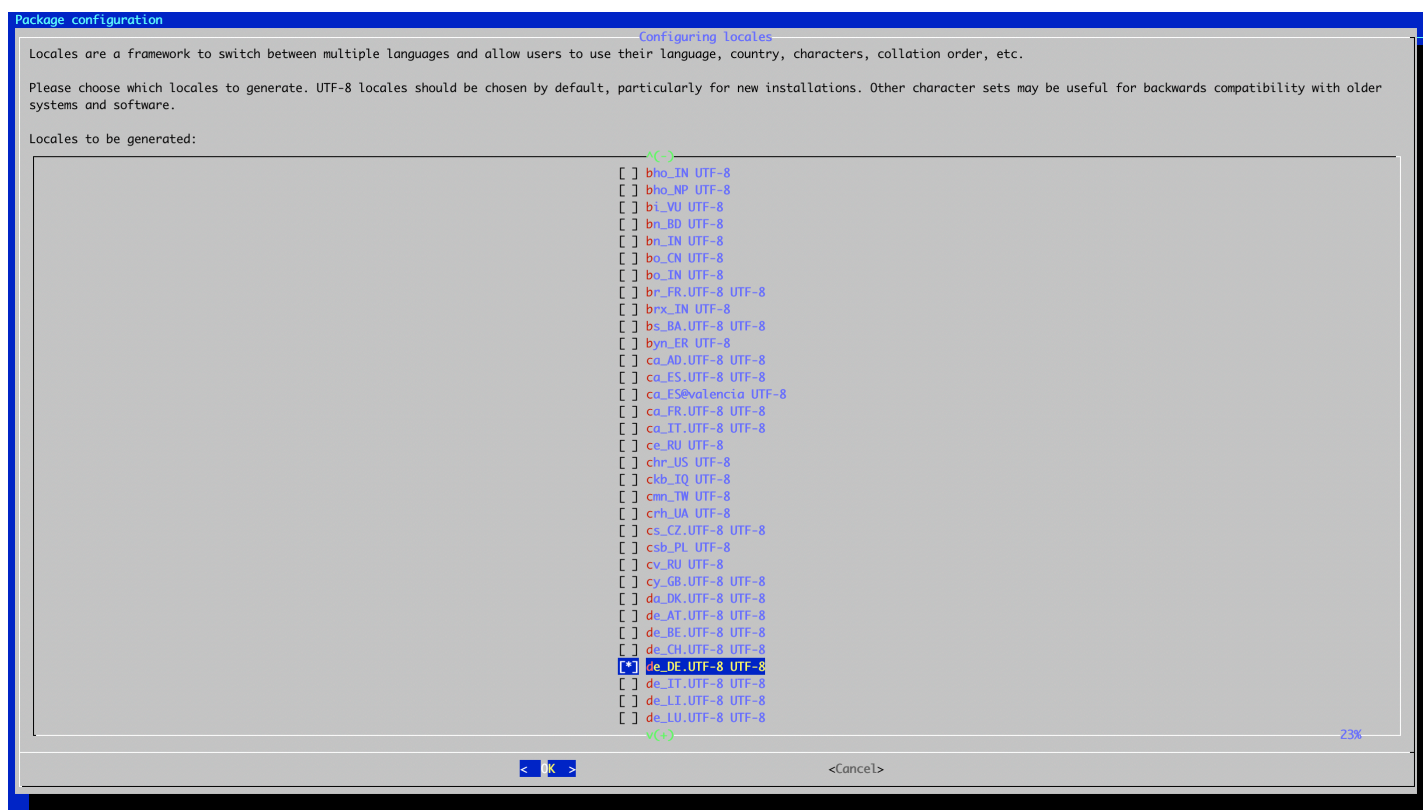


Abbildung 3. Installation / Hinzufügen Sprachpakete

Im darauf folgenden Screen muss im Anschluss die Standard Locale ausgewählt werden:

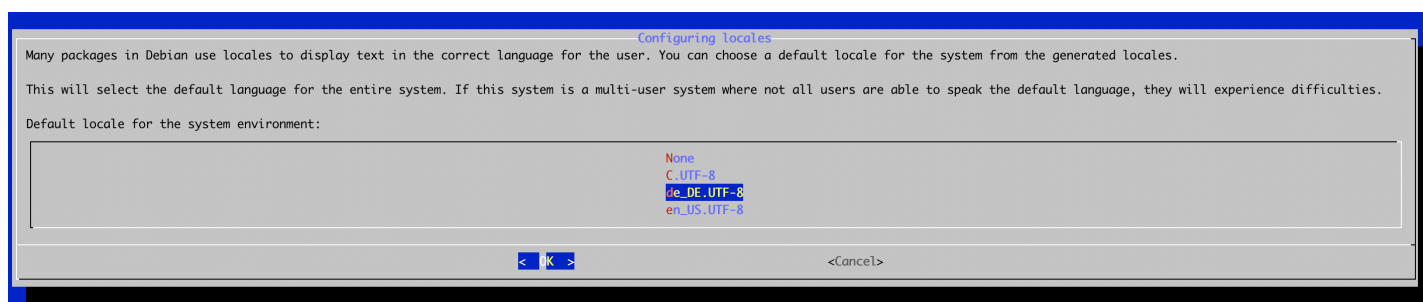


Abbildung 4. Auswahl Default

Sobald der Befehl `ansible --version` erneut ausgeführt wird, sollte der Fehler nicht mehr auftreten und einige Versionsinformationen in Bezug auf Ansible angezeigt werden. Eine beispielhafte Ausgabe kann folgendermaßen aussehen:

```
ansible [core 2.14.18]
  config file = None
  configured module search path = ['/root/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python3/dist-packages/ansible
  ansible collection location = /root/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/bin/ansible
  python version = 3.11.2 (main, Nov 30 2024, 21:22:50) [GCC 12.2.0] (/usr/bin/python3)
  jinja version = 3.1.2
  libyaml = True
```

TERMINAL

Auffällig ist der punkt `config file = None`. Hier sollte eigentlich der Pfad zur entsprechenden Konfiguration von Ansible stehen. Diese Konfiguration kann allerdings leicht angelegt werden:

```
mkdir /etc/ansible
touch /etc/ansible/ansible.cfg
```

TERMINAL

Als nächsten Schritt kann im entsprechenden Home-Verzeichnis des Users ein neues Verzeichnis `ansible_quickstart` erstellt werden:

```
mkdir ansible_quickstart && cd ansible_quickstart
```

TERMINAL

Ansible hat als Feature, dass auch innerhalb eines entsprechenden Verzeichnisses eine Konfiguration angelegt werden kann. Das wird im Verzeichnis `ansible_quickstart` mit folgendem Befehl durchgeführt:

```
touch ansible.cfg
```

TERMINAL

Erstellung eines Inventory

Die Managed Nodes werden in Ansible in zentralen Dateien organisiert, die Ansible mit Systeminformationen und Netzwerkstandorten versorgen. Mit einem Inventory kann Ansible eine große Anzahl von Hosts mit einem einzigen Befehl verwalten. (vgl. [ANS3])

Der Zugriff von Ansible aus auf die Managed Nodes erfolgt über SSH. Dazu muss der öffentliche SSH-Schlüssel des Control Nodes auf den Managed Nodes hinterlegt werden. (vgl. [ANS3])

Kopieren des SSH-Schlüssels

SSH-Schlüssel können relativ einfach von einem Host auf einen anderen Host kopiert werden. Hierfür wird der Befehl `ssh-copy-id` verwendet. Der Befehl wird wie folgt aufgerufen:

```
ssh-copy-id <USER>@<IP>
```

TERMINAL

Um das Vorhaben durchzuführen wird zunächst ein neuer SSH-Schlüssel auf dem Control Node generiert:

```
ssh-keygen
```

TERMINAL

Um im Anschluss den neuen Schlüssel zu kopieren, kann nachfolgender Befehl ausgeführt werden:

```
ssh-copy-id <USER>@<IP>
```

TERMINAL

Die Anlage eines Inventory

Das Inventory wird in der Regel in einer Datei `inventory.ini` oder `inventory.yaml` abgespeichert:

```
touch inventory.ini
```

TERMINAL

Innerhalb dieser Datei werden jetzt sämtliche Managed Nodes definiert, die von Ansible aus verwaltet werden sollen. Ein grundlegender Aufbau für ein Inventory kann folgendermaßen aussehen:

```
[myhosts]
10.0.147.1
10.0.147.2
10.0.147.3
10.0.147.4
10.0.147.5
```

INI

In `[]` Klammern steht dabei immer der Name der Gruppe, zu der die entsprechenden Managed Nodes gehören. Mit Hilfe von Gruppen kann unterschieden werden, welche Hosts zu welchen Zeiten und zu welchem Zweck verwaltet werden. Ansible kennt im Standard zwei Gruppen: (vgl. [ANS3])

- `all` :
Egal, ob ein Node innerhalb einer Gruppe steht oder nicht, `all` kennt alle Hosts, die innerhalb des Inventory stehen.
- `ungrouped` :
Alle Hosts, die nicht in einer Gruppe stehen, werden in dieser Gruppe zusammengefasst.

Ein etwas komplexeres Inventory könnte wie folgt aussehen:

```
mail.example.com

[webservers]
foo.example.com
bar.example.com

[dbservers]
one.example.com
two.example.com
three.example.com
```

INI

Natürlich ist es auch möglich, dass ein Managed Node Bestandteil mehrerer Gruppen ist. In diesem Fall wird der Managed Node in den entsprechenden Gruppen aufgeführt. Als Beispiel könnte man zum Beispiel einen produktiven Webserver heranziehen, der in einem Rechenzentrum in München steht. Dieser könnte in den Gruppen `[prod]`, `[munich]` und `[webservers]` aufgeführt werden. Gruppen können dabei folgende Informationen beinhalten:

- Was:
Eine Anwendung, ein Stack oder ein Microservice (zum Beispiel Datenbankserver, Webserver usw.).
- Wo:
Ein Rechenzentrum oder eine Region, um mit lokalen DNS, Speicher usw. zu kommunizieren (zum Beispiel Ost, West).
- Wann:
Die Entwicklungsstufe, um Tests auf Produktionsressourcen zu vermeiden (zum Beispiel Prod, Test).

Um ein entsprechendes Inventory zu verifizieren, ob Ansible es auch richtig interpretiert, kann der Befehl `ansible-inventory` verwendet werden. Dieser Befehl gibt das Inventory in einer JSON-Struktur aus:

```
ansible-inventory -i inventory.ini --list
```

TERMINAL

Die Parameter haben folgende Bedeutung:

- `-i` :
Der Pfad zur Inventory-Datei.
- `--list` :
Gibt das Inventory in einer JSON-Struktur aus.

`.ini` oder `.yaml`

Um ein Inventory aufzubauen, können entweder `.ini` oder `.yaml` Dateien verwendet werden. In den meisten Fällen, so wie auch im obigen Beispiel, werden `.ini` Dateien genutzt, da sie eine geringe Anzahl von verwalteten Knoten einfach und leicht zu lesen sind. (vgl. [ANS3])

Sobald allerdings die Anzahl zu verwalten Knoten zunimmt, wird die Erstellung eines Inventars im `.yaml` Format eine sinnvolle Option. Das aufgeführte Beispiel kann dementsprechend auch in einer `.yaml` Datei abgebildet werden:

```
myhosts:
  hosts:
    my_host_01:
      ansible_host: 10.0.147.1
    my_host_02:
      ansible_host: 10.0.147.2
    my_host_03:
      ansible_host: 10.0.147.3
    my_host_04:
      ansible_host: 10.0.147.4
    my_host_05:
      ansible_host: 10.0.147.5
```

YAML

Die Nutzung von Metagruppen

Innerhalb eines Inventory können auch Metagruppen definiert werden. Metagruppen sind Gruppen, die andere Gruppen zusammenfassen. Mehr oder weniger werden somit Eltern-Kind-Beziehungen dargestellt! Ein Beispiel für eine Metagruppe könnte wie folgt aussehen:

```
cust1_dev:
  hosts:
    cust1_dev1:
      ansible_host: 10.0.147.1
    cust1_dev2:
      ansible_host: 10.0.147.2
    cust1_dev3:
      ansible_host: 10.0.147.3

cust1_test:
  hosts:
    cust1_test1:
      ansible_host: 10.0.147.4

cust1_prod:
  hosts:
    cust1_prod1:
      ansible_host: 10.0.147.5
    cust1_prod2:
      ansible_host: 10.0.147.6

cust2_dev:
  hosts:
    cust2_dev1:
      ansible_host: 10.0.147.7

cust2_test:
  hosts:
    cust2_test1:
      ansible_host: 10.0.147.8
    cust2_test2:
      ansible_host: 10.0.147.9

cust2_prod:
  hosts:
    cust2_prod1:
      ansible_host: 10.0.147.10
    cust2_prod2:
      ansible_host: 10.0.147.11
    cust2_prod3:
      ansible_host: 10.0.147.12

cust3_dev:
  hosts:
    cust3_dev1:
      ansible_host: 10.0.147.13

cust3_test:
  hosts:
    cust3_test1:
      ansible_host: 10.0.147.14

cust3_prod:
  hosts:
    cust3_prod1:
      ansible_host: 10.0.147.15

cust4_dev:
  hosts:
    cust4_dev1:
      ansible_host: 10.0.147.16
    cust4_dev2:
      ansible_host: 10.0.147.17
    cust4_dev3:
      ansible_host: 10.0.147.18

cust4_test:
```



```
hosts:
  cust4_test1:
    ansible_host: 10.0.147.19
  cust4_test2:
    ansible_host: 10.0.147.20

cust4_prod:
  hosts:
    cust4_prod1:
      ansible_host: 10.0.147.21
    cust4_prod2:
      ansible_host: 10.0.147.22

cust1:
  children:
    cust1_dev:
    cust1_test:
    cust1_prod:

cust2:
  children:
    cust2_dev:
    cust2_test:
    cust2_prod:

cust3:
  children:
    cust3_dev:
    cust3_test:
    cust3_prod:

cust4:
  children:
    cust4_dev:
    cust4_test:
    cust4_prod:

others:
  hosts:
    others1:
      ansible_host: 10.0.147.23
    others2:
      ansible_host: 10.0.147.24
    others3:
      ansible_host: 10.0.147.25
    others4:
      ansible_host: 10.0.147.26

customers:
  children:
    cust1:
    cust2:
    cust3:
    cust4:
    others:
```

Ein etwas kürzeres Beispiel mit insgesamt drei Managed Nodes könnte wie folgt aussehen:

```

cust1_dev:
  hosts:
    cust1_dev1:
      ansible_host: 10.0.146.51

cust1_test:
  hosts:
    cust1_test1:
      ansible_host: 10.0.146.116

cust1_prod:
  hosts:
    cust_1prod1:
      ansible_host: 10.0.146.117

devs:
  hosts:
    dev1:
      ansible_host: 10.0.146.51

tests:
  hosts:
    test1:
      ansible_host: 10.0.146.116

prods:
  hosts:
    prod1:
      ansible_host: 10.0.146.117

cust1:
  children:
    cust1_dev:
    cust1_test:
    cust1_prod:

customers:
  children:
    cust1:

master:
  children:
    customers:
    devs:
    tests:
    prods:

```

Verbindung testen mit einem einfachen Beispiel

Um die Verbindung zu den einzelnen Managed Nodes zu testen, kann der Befehl `ansible` in Kombination mit dem Modul `ping` genutzt werden. Der Befehl dazu sieht folgendermaßen aus:

```
ansible customers -m ping -i inventory.yaml
```

TERMINAL

Aus einem komplexeren Inventory können natürlich auch einzelne Hosts verwendet werden:

```
ansible cust1 -m ping -i inventory.yaml
```

TERMINAL

Um auf einem verwalteten Managed Node im Anschluss ein Kommando auszuführen, kann der Befehl `ansible` in Kombination mit dem Modul `command` genutzt werden. Der Befehl dazu sieht folgendermaßen aus:

```
ansible cust1_dev -m command -a '<Command>' -i inventory.yaml
```

Nützliche Parameter

User wechseln und Rechte anpassen

Häufig benötigt man auf einem Managed Node allerdings auch die Rechte von Root. Im Inventory können hierfür zwei Parameter gesetzt werden, welche den aktuellen User auf das Level von Root heben:

YAML

```
...
ansible_become: true
ansible_become_pass: <Password>
...
```

Möchte man allerdings direkt mit einem anderen User wie zum Beispiel `root` arbeiten, gibt es auch hierfür entsprechende Parameter:

YAML

```
...
ansible_user: <UserToExecute>
...
```

Hierzu müssen allerdings einige vorbereitende Schritte durchgeführt werden:

- Der User `root` muss für eingehende SSH-Verbindungen auf dem Managed Node freigeschalten sein. Bedeutet, dass die Option `PermitRootLogin` innerhalb von `/etc/ssh/sshd_config` auf `yes` gesetzt sein muss. Neben dieser Eigenschaft sollten unbedingt nicht die folgenden Eigenschaften konfiguriert werden:
 - `PubkeyAuthentication yes`
 - `PasswordAuthentication no`
- SSH Daemon neustarten via `systemctl restart sshd`.
- Der User `root` braucht ein Passwort auf dem Managed Node. Das kann gesetzt werden via `passwd`.
- Der User `root` muss in der Lage sein, sich ohne Passwort auf dem Managed Node einzuloggen. Hierfür muss der öffentliche SSH-Schlüssel des Control Nodes auf den Managed Nodes hinterlegt werden. Das passiert allerdings auch vom aktuellen User aus, der Ansible ausführt. Kopiert werden kann erneut via `ssh-copy-id`.

Verbindung über einen anderen Port

Im Standard geht Ansible auch davon aus, dass die Kommunikation über den Port `22` stattfindet. Sollte auf einem entfernten Rechner allerdings eine andere Konfiguration vorgenommen und der Port z. B. auf `2222` geändert worden sein, kann dies im Inventory ebenfalls angepasst werden:

YAML

```
...
ansible_port: <PortToUse>
...
```

Variablen

Wenn man das Inventory in Gruppen unterteilt, werden häufig für die einzelnen Hosts Variablen benötigt, welche sich ebenfalls überschneiden können. Aus diesem Grund gibt es in Ansible die Möglichkeit, dass entsprechende Variablen abgelegt werden können. Man unterscheidet dabei zwischen zwei Arten von Variablen:

- **Gruppen-Variablen:**
Diese Variablen werden für eine Gruppe definiert und gelten für alle Hosts innerhalb der Gruppe.

- **Host-Variablen:**

Diese Variablen werden direkt für einen Host definiert und gelten nur für den entsprechenden Host.

Gruppen-Variablen

Als Ausgangsbasis wird nachfolgendes Inventory herangezogen:

YAML

```
second_sub1:
  hosts:
    second_sub1_1:
      ansible_host: 10.0.147.31
      ansible_user: root
    second_sub1_2:
      ansible_host: 10.0.147.32
      ansible_user: root

second_sub2:
  hosts:
    second_sub2_1:
      ansible_host: 10.0.147.33
      ansible_become: true
      ansible_become_pass: tester123
    second_sub2_2:
      ansible_host: 10.0.147.34
      ansible_become: true
      ansible_become_pass: tester123

sub1:
  children:
    second_sub1:
    second_sub2:

sub2:
  hosts:
    sub2_1:
      ansible_host: 10.0.147.35
      ansible_port: 2222

master:
  children:
    sub1:
    sub2:
```

Wie man sieht, wechseln die Hosts `second_sub1_1` und `second_sub1_2` den User auf `root`, während die Hosts `second_sub2_1` und `second_sub2_2` den User auf `root` heben und das Passwort auf `tester123` setzen. Der Host `sub2_1` wechselt den Port auf `2222`.

Für die Hosts `second_sub1_1` und `second_sub1_2` sowie für die Hosts `second_sub2_1` und `second_sub2_2` können die entsprechenden Eigenschaften nur ein einziges Mal in Form von Gruppenvariablen definiert werden:

```

second_sub1:
  hosts:
    second_sub1_1:
      ansible_host: 10.0.147.31
    second_sub1_2:
      ansible_host: 10.0.147.32
  vars:
    ansible_user: root

second_sub2:
  hosts:
    second_sub2_1:
      ansible_host: 10.0.147.33
      ansible_become: true
      ansible_become_pass: tester123
    second_sub2_2:
      ansible_host: 10.0.147.34
      ansible_become: true
      ansible_become_pass: tester123
  vars:
    ansible_become: true
    ansible_become_pass: tester123

sub1:
  children:
    second_sub1:
    second_sub2:

sub2:
  hosts:
    sub2_1:
      ansible_host: 10.0.147.35
      ansible_port: 2222

master:
  children:
    sub1:
    sub2:

```

Möchte man für alle Hosts entsprechende Variablen setzen, kann dies auch direkt unterhalb des `all` -Schlüssels erfolgen:

```

...
all:
  vars:
    ansible_ssh_user: sheitzer
...

```

Host-Variablen

Host-Variablen werden direkt unterhalb des entsprechenden Hosts definiert. Ein Beispiel hierfür ist zum Beispiel der Host `sub2_1`, welcher den Port auf `2222` setzt:

```

...
sub2:
  hosts:
    sub2_1:
      ansible_host: 10.0.147.35
      ansible_port: 2222
...

```

Anpassungen in der `ansible.cfg`

Testet man ein Modul mit Hilfe von Ansible, dann musste bisher immer das Inventory mit angegeben werden. Dies kann jedoch auch in der Konfigurationsdatei `ansible.cfg` hinterlegt werden. Hierzu wird die Datei `ansible.cfg` mit einem Editor geöffnet und die folgenden Zeilen hinzugefügt:

```
[defaults]
inventory = /path/to/inventory
```

CFG

Im Anschluss kann z. B. das Modul `ping` ohne Angabe des Inventars aufgerufen werden:

```
ansible second_sub1_1 -m ping
```

TERMINAL

Erstellung eines Playbooks

Playbooks sind eigentlich nur Skripte, welche mit der Auszeichnungssprache YAML erstellt werden. In ihnen wird definiert, in welchem Zustand sich Systeme befinden sollten. Ansible verwendet diese Playbooks und wendet diese zur Auslieferung und Konfiguration auf die Managed Nodes an. (vgl. [ANS4])

Zu den Grundbestandteilen der Playbooks zählen dabei:

- **Playbook:**
Eine Liste von Schritten, die die Reihenfolge definiert, in der Ansible Operationen durchführt, um ein Gesamtziel zu erreichen. (vgl. [ANS4])
- **Play:**
Ein Abschnitt eines Playbooks, der eine Gruppe von Aufgaben definiert, die auf einer Gruppe von Hosts ausgeführt werden sollen. (vgl. [ANS4])
- **Task:**
Ein Verweis auf ein einzelnes Modul, das die von Ansible durchgeführten Operationen definiert. (vgl. [ANS4])
- **Module:**
Eine Einheit von Code oder Binärdatei, die Ansible auf den verwalteten Managed Nodes ausführt. In Ansible sind Module in Sammlungen gruppiert und haben einen vollständig qualifizierten Sammlungsnamen (FQCN). (vgl. [ANS4])

Ein einfaches Playbook

Um ein einfaches Playbook zu erstellen, kann im Ordner `ansible_quickstart` eine neue Datei mit nachfolgendem Namen angelegt werden:

```
touch playbook.yml
```

TERMINAL

In dieser Datei kann dann das folgende Playbook erstellt werden:

```
- name: My first play
  hosts: myhosts
  tasks:
    - name: Ping my hosts
      ansible.builtin.ping:

    - name: Print message
      ansible.builtin.debug:
        msg: Hello world
```

TERMINAL

Das Playbook selbst kann im Anschluss relativ einfach mit diesem Befehl getestet werden:

```
ansible-playbook -i inventory.yaml playbook.yaml
```

Erläuterung

- **name :**
Hierbei handelt es sich um den Namen des `Plays`, der ausgeführt werden sollte.
- **hosts :**
Mit dieser Eigenschaft wird die Gruppe der Hosts angegeben, auf denen das `Play` ausgeführt werden soll.
- **tasks :**
Die `tasks` sind die Auflistung der einzelnen Aufgaben, die im `Play` ausgeführt werden sollen.
 - **name :**
Der Name der Aufgabe, die ausgeführt werden soll. Nicht nur `Plays` haben folglich eigene Namen, sondern auch die `Tasks`.
 - **Spezifikation der Moduls:**
Jeder `task` nutzt ein entsprechendes Modul, damit Aufgaben ausgeführt werden können. Im Beispiel werden zum Beispiel die beiden Module `ansible.builtin.ping` und `ansible.builtin.debug` verwendet.

Farbschema

Sobald ein Befehl in Ansible ausgeführt wird und man die entsprechende Rückgabe bzw. das Log des Befehls betrachtet, wird man immer wieder unterschiedliche Statusfarben erkennen:

- **Rot:** Befehl wurde ausgeführt, allerdings kam es zu einem Fehler.
- **Gelb:** Befehl wurde erfolgreich ausgeführt. Auf dem entsprechenden Managed Node kam es dabei zu einer Veränderung des Zustandes.
- **Grün:** Der Befehl wurde ebenfalls erfolgreich ausgeführt, allerdings gab es auf dem Managed Node keinerlei Veränderung.

Module in Ansible

Module in Ansible sind die eigentlichen Arbeitspferde. Sie führen die eigentlichen Aufgaben aus. Ein Modul kann entweder auf dem Control Node oder auf dem Managed Node ausgeführt werden. Die Module sind in Python geschrieben und können entweder von Ansible selbst oder von der Community bereitgestellt werden. Die Module sind in der Regel sehr spezialisiert und führen nur eine bestimmte Aufgabe aus. Im folgenden Teilkapitel werden einige der Module näher beschrieben und erläutert: (vgl. [ANS3])

setup Module

Beim `setup` Module handelt es sich um ein spezielles Modul, welches Informationen über den Managed Node sammelt und zurückgibt. Dieses Modul wird automatisch aufgerufen, wenn ein Playbook ausgeführt wird. Es ist nicht notwendig, dieses Modul explizit aufzurufen. Die Ausgabe des `setup` Moduls kann mit dem Befehl `ansible` und dem Modul `setup` abgerufen werden:

```
ansible second_sub1_1 -m setup -i inventory.yaml
```

file Module

Das `file` Modul wird verwendet, um Dateien und Verzeichnisse auf dem Managed Node zu erstellen, zu ändern oder zu löschen. Das Modul kann auch verwendet werden, um die Berechtigungen von Dateien und Verzeichnissen zu ändern. Das Modul `file` kann mit dem folgenden Befehl aufgerufen werden:

```
ansible -m file -a 'path=/tmp/test state=touch' second_sub1_1 -i inventory.yaml
```

Durch den Befehl wird im Verzeichnis `/tmp` auf dem Managed Node eine leere Datei mit dem Namen `test` erstellt. Die Ausgabe des Befehls sieht wie folgt aus:

```
second_sub1_1 | CHANGED => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": true,
  "dest": "/tmp/test",
  "gid": 0,
  "group": "root",
  "mode": "0644",
  "owner": "root",
  "size": 0,
  "state": "file",
  "uid": 0
}
```

Man erhält in Bezug auf die Ausgabe auch Information über den Besitzer, die Gruppe, die Berechtigungen und die Größe der Datei.

Möchte man zum Beispiel eine Datei auf dem Managed Node löschen, muss der `state` innerhalb des Befehls auf `absent` gesetzt werden. Im Endeffekt gibt es für den `state` die folgenden Optionen:

- `absent` :
Löscht die Datei oder das Verzeichnis.
- `directory` :
Erstellt ein Verzeichnis.
- `file` :
Ohne weitere Zusätze wird hier lediglich Information zur spezifizierten Datei ausgegeben. Möchte man allerdings z. B. die Rechte einer Datei ändern, so kann das mit dem Zusatz `mode` durchgeführt werden.
- `touch` :
Erstellt eine leere Datei.
- `link` :
Erstellt einen symbolischen Link.
- `hard` :
Erstellt einen Hardlink.

copy Module

Wie der Name vermuten lässt, kann das `copy` Module verwendet werden, damit Dateien zum Beispiel von einem Control Node auf einen Managed Node kopiert werden können. Der Befehl dazu sieht folgendermaßen aus:

```
ansible -m copy -a 'src=/tmp/file dest=/tmp/file' second_sub1_1
```

Möchte man, dass eine Datei von einem Managed Node auf eine andere Datei kopiert wird, kann als zusätzlicher Parameter `remote_src=yes` angegeben werden. Der Befehl dazu sieht folgendermaßen aus:

```
ansible -m copy -a 'remote_src=yes src=/tmp/file dest=/tmp/file2' second_sub1_1
```


command Module

Bereits in vorherigen Kapiteln wurde das `command` Module verwendet. Immer dann, wenn auf einem Managed Node ein herkömmliches Kommando ausgeführt werden sollte, kann dieses Modul verwendet werden:

```
ansible -m command -a 'uptime' second_sub1_1
```

TERMINAL

Du beachten ist allerdings, dass das `command` Module keine direkte Shell darstellt! Folglich können spezifische Funktionen wie `|`, `>`, `>>`, `&&` etc. nicht verwendet werden. Sollte dies dennoch notwendig sein, kann das Modul `shell` verwendet werden.

Weiterhin ist das Modul auch der Standard. Es muss demnach nicht extra mit dem Parameter `-m` angegeben werden. Um ein Verzeichnis zum Beispiel zu erstellen, sieht der Befehl folgendermaßen aus:

```
ansible -a 'mkdir /tmp/folder1' second_sub1_1
```

TERMINAL

fetch Module

Im Gegensatz zum `copy` Module, wird das `fetch` Module benutzt, damit Dateien von einem oder mehreren Managed Nodes auf den entsprechenden Control Node zu übertragen. Ein einfaches Beispiel hierfür sieht folgendermaßen aus:

```
ansible -m fetch -a "src=/etc/hosts dest=./files/ flat=yes"
```

TERMINAL

In diesem Beispiel wird die Datei `/etc/hosts` von allen Managed Nodes auf den Control Node in das Verzeichnis `./files/` kopiert. Der Parameter `flat=yes` sorgt dafür, dass die Dateien nicht in einem Unterordner abgelegt werden, sondern direkt im angegebenen Verzeichnis. Bei der Option `no` wird der Dateiname des Managed Nodes als Unterordner verwendet.

Playbooks in Ansible

YAML

YAML ist die Abkürzung für `Yet Another Markup Language` bzw. für `YAML Ain't Markup Language`. Bei der vermeintlichen Auszeichnungssprache handelt es sich mehr um eine sehr einfache, von Menschen lesbare Datenserialisierungssprache als Auszeichnungssprache. Ihr Hauptanwendungsgebiet liegt daher auch eher in der Auszeichnung von Daten als von entsprechenden Dokumenten. (vgl. [YAM1])

Zu den wesentlichen Merkmale von YAML gehören:

- **Lesbarkeit:**
YAML ist für Menschen leicht lesbar und schreibbar. Die Syntax ist einfach und intuitiv.
- **Hierarchische Struktur:**
YAML unterstützt eine hierarchische Struktur. Somit können komplexere Datenstrukturen dargestellt werden.
- **Datenserialisierung:**
YAML kann Daten serialisieren und deserialisieren. Somit werden die Dateien sehr häufig genutzt, damit Daten zwischen verschiedenen Programmen ausgetauscht werden können.
- **Whitespace:**
YAML verwendet Leerzeichen zur Strukturierung. Somit ist die Einrückung von Blöcken sehr wichtig.

Hauptverwendung von YAML ist innerhalb von Konfigurationsdateien und in der Datenserialisierung. (vgl. [YAM1])

Die Bestandteile des Playbooks

Das Playbook ist das Herzstück von Ansible. In ihm können mehrere Aufgaben definiert werden, die auf entsprechenden Managed Nodes ausgeführt werden sollen, damit im Anschluss ein System auf einen gewünschten Zustand gebracht wird. Ein Playbook selbst besteht aus mehreren Bestandteilen. Diese sind: (vgl. [ANS7])

- **Name:**

Der Name des Playbooks. Dieser sollte möglichst aussagekräftig sein, damit auch nach längerer Zeit noch klar ist, was das Playbook bewirkt. (vgl. [ANS7])

- **Hosts:**

Die Managed Nodes, auf denen das Playbook ausgeführt werden soll. Hierbei kann es sich um einzelne Hosts, Hostgruppen oder auch um alle Hosts handeln. Hier spielt die Gruppenstruktur des Inventories eine wichtige Rolle. (vgl. [ANS7])

- **Vars:**

Variablen, die innerhalb des Playbooks genutzt werden können. Diese können entweder global oder auf Hostgruppen bezogen sein. Variablen können zum Beispiel für die Konfiguration von Diensten oder für die Definition von Dateipfaden genutzt werden. (vgl. [ANS7])

- **Tasks:**

Die eigentlichen Aufgaben, die auf den Managed Nodes ausgeführt werden sollen. Ein Task besteht aus einem oder mehreren Modulen, die auf den Managed Nodes ausgeführt werden. (vgl. [ANS7])

- **Handlers:**

Handler werden nur dann ausgeführt, wenn sie für die z. B. Benachrichtigung eines anderen Dienstes benötigt werden. Ein Handler wird nur dann ausgeführt, wenn ein Task ihn explizit aufruft. Eine Änderung an einem System bewirkt zum Beispiel, dass ein entsprechender Handler auch ausgeführt wird. (vgl. [ANS7])

- **Roles:**

Rollen können in ein Playbook eingebunden werden um somit auch die Wiederverwendbarkeit von Playbooks zu erhöhen. (vgl. [ANS7])

Im den folgenden Teilkapiteln werden mehrere unterschiedliche Playbooks aufgebaut und erläutert. Dabei wird auch jeweils auf die einzelnen Bestandteile näher eingegangen!

motd play

motd ist die Abkürzung für Message of the Day. Es handelt sich hierbei um eine Datei, die beim Anmelden an einem System angezeigt wird. Diese Datei kann mit dem motd Play angepasst werden. Um dieses Playbook zu erstellen, wird ein neuer Ordner angelegt:

```
mkdir motd_play
```

TERMINAL

Innerhalb des Ordners werden die nachfolgenden Dateien angelegt:

- `ansible.cfg`
- `hosts.yaml`
- `motd`
- `motd_playbook.yaml`

`ansible.cfg`

```
nano ansible.cfg
```

TERMINAL

In der Datei `ansible.cfg` wird der Pfad zum Inventory angegeben:

```
[defaults]
inventory = ./hosts.yaml
```

CFG

hosts.yaml

```
nano hosts.yaml
```

TERMINAL

In der Datei `hosts.yaml` werden die Managed Nodes definiert:

```
ansible_nodes:
  hosts:
    ansible_node:
      ansible_host: 10.0.147.X
```

YAML

motd

```
nano motd
```

TERMINAL

Innerhalb dieser Datei kann eine Nachricht hinterlegt werden, die beim Anmelden an einem System angezeigt wird:

```
Herzlich willkommen auf meinem Ansible Managed Node!
```

TERMINAL

motd_playbook.yaml

```
nano motd_playbook.yaml
```

TERMINAL

Der beispielhafte Inhalt des Playbooks könnte folgendermaßen aussehen:

```
- name: MotD Play (1)
  hosts: ansible_nodes (2)
  tasks: (3)
    - name: Copy motd file (4)
      copy: (5)
        src: motd (6)
        dest: /etc/motd (7)
```

YAML

IMPORTANT

Ein `-` stellt in YAML immer eine Liste dar. Auch ein Playbook ist mehr oder weniger eine Liste von mehreren einzelnen Plays! Daher muss auch ein Play immer mit einem `-` beginnen.

- **(1):**

Hier wird der Name des Plays angegeben. Das Schlüsselwort hierfür ist `name`. Im Playbook selbst können ja mehrere Plays vorhanden sein.

- **(2):**

Hier werden die Managed Nodes angegeben, auf welche das Playbook Anwendung finden soll. Das Schlüsselwort hierfür ist `hosts`.

- **(3):**
Mit Hilfe des Schlüsselworts `tasks` werden die einzelnen Aufgaben definiert, die im Play ausgeführt werden sollen. Es handelt sich hierbei um erneut um eine Liste, daher wird wieder ein `-` benötigt!
- **(4):**
Jede Aufgabe die ausgeführt wird, sollte ebenfalls zur besseren Orientierung einen Namen haben. Dieser wird erneut mit dem Schlüsselwort `name` definiert.
- **(5):**
Nach dem Namen des Tasks wird das eigentliche Modul definiert, welches ausgeführt werden sollte. In diesem Fall wird das Modul `copy` verwendet.
- **(6) / (7):**
Das Modul `copy` benötigt zwei Parameter, die mit `src` und `dest` definiert werden. `src` gibt den Pfad zur Quelldatei an, `dest` den Pfad zum Zielort.

Die Ausführung des Playbooks

Um das Playbook endgültig ausführen zu können, muss das Kommando `ansible-playbook` verwendet werden:

```
ansible-playbook motd_playbook.yaml
```

TERMINAL

Die Ausgabe der ersten Ausführung

```
PLAY [MotD Play] *****

TASK [Gathering Facts] *****
ok: [ansible_node]

TASK [Copy motd file] *****
fatal: [ansible_node]: FAILED! => {"changed": false, "checksum": "38179d1e5b804bbe3bb77c8051adda8d9ff09066",
"msg": "Destination /etc not writable"}

PLAY RECAP *****
ansible_node1      : ok=1    changed=0    unreachable=0    failed=1    skipped=0    rescued=0
ignored=0
```

TERMINAL

Die Ausführung des Playbooks schlägt fehl! Grund dafür ist, dass der Zielort `/etc` nicht beschreibbar ist. Um das Problem zu beheben, kann das Playbook wie folgt angepasst werden:

```
...
user: root
...
```

YAML

Die Eigenschaft `user` muss dabei in der Grunddefinition des Plays spezifiziert werden!

Die Ausgabe der zweiten Ausführung

```
PLAY [MotD Play] *****

TASK [Gathering Facts] *****
ok: [ansible_node]

TASK [Copy motd file] *****
changed: [ansible_node]

PLAY RECAP *****
ansible_node      : ok=2    changed=1    unreachable=0    failed=0    skipped=0    rescued=0
ignored=0
```

TERMINAL

Wie auch schon bei der ersten Ausführung, bemerkt man auch bei der zweiten Ausführung, dass, obwohl lediglich ein einziger Task im Playbook definiert wurde, zwei Tasks ausgeführt wurden:

- Gathering Facts
- Copy motd file

Bei dem zweiten Task handelt es sich um den eigens erstellten Task innerhalb des Playbooks. Der Task `Gathering Facts` allerdings wird von Ansible automatisch hinzugefügt und sammelt Informationen über die Managed Nodes. Hinter diesem Task befindet sich das Modul `setup`, welches die entsprechenden Informationen sammelt.

Aufgabe

Erstellen Sie da oben aufgeführte Playbook und führen Sie dieses auf Ihrem eigenen Managed Node (`10.0.147.X`) aus. Beachten Sie dabei alle notwendigen Schritte, die erforderlich sind, damit Sie vom Control Node auf den Managed Node zugreifen können!

Lösung

Aufgaben auf `10.0.147.X`

Zunächst muss ein entsprechender User auf dem Managed Node erstellt werden:

```
useradd -m -s /bin/bash <YourUser>

passwd <YourUser>
```

SHELL

Anschließend muss die SSH Konfiguration angepasst werden, damit eine Verbindung von `10.0.146.X` auf `10.0.147.X` mit einem Schlüssel stattfinden kann (betroffen hierbei sind die User `root` und der neu angelegte Benutzer):

```
...
PermitRootLogin yes
PubkeyAuthentication yes
PasswordAuthentication yes
...
```

SHELL

Sobald die Einstellungen vorgenommen worden sind, muss der Deamon neugestartet werden:

```
systemctl restart sshd
```

SHELL

NOTE

Sobald die Schlüsselpaare übertragen worden sind, gehört die Konfiguration wieder angepasst!

Die Konfiguration nach der Übertragung der Schlüssel ist nachfolgende:

```
...
PermitRootLogin prohibit-password
PubkeyAuthentication yes
PasswordAuthentication no
...
```

SHELL

Weiterhin muss auf dem entsprechenden System `python` installiert sein:

```
apt-get update && apt-get install python3
```

SHELL

Aufgaben auf 10.0.146.X

Auf dem Control Node müssen die Schlüssel aktuellen Users für den neu angelegten User und den Benutzer `root` auf `10.0.147.X` übertragen werden:

```
ssh-copy-id 10.0.147X
```

SHELL

```
ssh-copy-id root@10.0.147.X
```

Mit diesen vorbereitenden Schritten kann jetzt das Playbook erstellt und ausgeführt werden!

Abwandlung des Playbooks

Häufig gibt es auch den Fall, dass die `Message of the Day` nicht in einer separaten Datei abgelegt wird, sondern direkt im Playbook spezifiziert wird. Das kann mit dem Zusatz `content` erreicht werden. Das abgeänderte Playbook könnte wie folgt aussehen:

```
- name: MotD Play
  hosts: ansible_nodes
  user: root
  tasks:
    - name: Copy motd file
      copy:
        content: This is my new awesome Message of the Day!
        dest: /etc/motd
```

YAML

Es gibt allerdings auch die Möglichkeit, dass die `Message of the Day` mit Hilfe einer Variable spezifiziert wird. Hierfür gibt es das Schlüsselwort `vars`:

```
- name: MotD Play
  hosts: ansible_nodes
  user: root
  vars:
    motd_content: Message of the Day as Variable!
  tasks:
    - name: Copy motd file
      copy:
        content: "{{ motd_content }}"
        dest: /etc/motd
```

YAML

Die Variable `motd_content` ist hier mit einem entsprechenden Wert vorbelegt. Die Variable kann allerdings auch beim Ausführen des Playbooks mit einem Inhalt übergeben werden. Hierfür wird der Parameter `-e` verwendet:

```
ansible-playbook motd_playbook.yaml -e 'motd_content="This is my new awesome Message of the Day! Passed as an argument"'
```

SHELL

Der Einsatz von Handlern

Handler werden ausgelöst, sobald ein entsprechender Task mit einem Notifier versehen wurde. Das Schlüsselwort hierfür ist `notify`. Die Handler selbst werden mit dem Schlüsselwort `handlers` definiert und sind eine Liste. Jedes Mal, wenn folglich ein Task erfolgreich ausgeführt worden ist und eine Änderung am System vorgenommen wurde, greift der Handler. Das ist vor allem für entsprechende Folgeaktionen interessant:

```
- name: MotD Play
  hosts: ansible_nodes
  user: root
  vars:
    motd_content: Message of the Day as Variable!
  tasks:
    - name: Copy motd file
      copy:
        content: "{{ motd_content }}"
        dest: /etc/motd
      notify: Print message
  handlers:
    - name: Print message
      debug:
        msg: Message of the Day was changed!
```

Soll ein Task ausgeführt werden?

Häufig hat man in einem Playbook mehrere verschiedene Tasks und natürlich auch mehrere Hosts spezifiziert. Es kann allerdings auch der Fall eintreten, dass ein Task nur dann ausgeführt werden soll, wenn ein bestimmter Host betroffen ist. Hierfür gibt es das Schlüsselwort `when` :

```
- name: MotD Play
  hosts: ansible_nodes
  user: root
  vars:
    motd_content: Message of the Day as Variable!
  tasks:
    - name: Copy motd file
      copy:
        content: "{{ motd_content }}"
        dest: /etc/motd
      notify: Print message
      when: ansible_distribution == 'Ubuntu'
  handlers:
    - name: Print message
      debug:
        msg: Message of the Day was changed!
```

Im aufgeführten Beispiel wird der Task nur dann ausgeführt, wenn die Distribution `Ubuntu` ist.

Variablen

Bereits im vorherigen Kapitel wurde die erste Verwendung von Variablen aufgezeigt. Innerhalb eines Playbooks gibt es immer die Möglichkeit mit Hilfe des Schlüsselwortes `vars` Variablen zu definieren und diese auch mit Werten vorzubelegen:

```
- name: MotD Play
  hosts: ansible_nodes
  user: root
  vars:
    motd_content: Message of the Day as Variable!
  tasks:
    - name: Copy motd file
      copy:
        content: "{{ motd_content }}"
        dest: /etc/motd
```

Der Zugriff auf die Variable erfolgt dabei über zwei geschweifte Klammern.

Für eine bessere Übersichtlichkeit der Variablen, können diese ebenfalls in Gruppen zusammengefasst werden:

YAML

```
- name: Play for using variables
  hosts: ansible_nodes
  vars:
    motd:
      content: Message of the Day as Variable!
  tasks:
    - name: Copy motd file
      copy:
        content: "{{ motd.content }}"
        dest: /etc/motd
```

In diesem Fall wird die Variable `motd` mit dem Inhalt `content` definiert. Der Zugriff auf die Variable erfolgt dann über `motd.content`. Ebenfalls wäre der Zugriff mit `{{ motd['content'] }}` möglich.

Es gibt auch die Möglichkeit, dass Variablen als Liste definiert werden:

YAML

```
- name: Play for using variables
  hosts: ansible_nodes
  user: root
  vars:
    list_of_variables:
      - var1
      - var2
      - var3
  tasks:
    - name: Print all variables
      debug:
        msg: "{{ list_of_variables }}"
    - name: Print first variable with bracket notation
      debug:
        msg: "{{ list_of_variables[0] }}"
    - name: Print second variable with dot notation
      debug:
        msg: "{{ list_of_variables.1 }}"
```

Das gleiche Ziel kann allerdings auch mit Hilfe eines Arrays erreicht werden. Die Syntax dafür sieht folgendermaßen aus:

YAML

```
...
vars:
  list_of_variables2:
    [var1, var2, var3]
...
```

Allgemein ist es allerdings besser, wenn die Variablen in eine separate Datei ausgelagert werden. Eine solche Datei könnte mit allen bisherigen Beispielen folgendermaßen aussehen:


```
motd_content: Message of the Day as Variable!
```

```
motd:
  content: Message of the Day as Variable!
```

```
list_of_variables:
- var1
- var2
- var3
```

```
list_of_variables2:
[var1, var2, var3]
```

Diese Datei kann nun mit dem Schlüsselwort `vars_files` in das Playbook eingebunden werden. Das Playbook würde dann wie folgt aussehen:

```
- name: Test for variables
  hosts: ansible_nodes
  vars_files:
    - vars.yaml
  tasks:
    - name: Read simple key
      debug:
        msg: "{{ motd_content }}"
    - name: Read nested variable
      debug:
        msg: "{{ motd.content }}"
    - name: Read variable from list with dot notation
      debug:
        msg: "{{ list_of_variables.0 }}"
    - name: Read variable from list with bracket notation
      debug:
        msg: "{{ list_of_variables[0] }}"
    - name: Print all variables from a list
      debug:
        msg: "{{ list_of_variables2 }}"
```

Usereingaben als Variablen nutzen

Es gibt in Ansible natürlich auch die Möglichkeit, dass der Benutzer während der Ausführung des Playbooks Variablen eingeben kann. Dies geschieht mit Hilfe des Schlüsselwortes `vars_prompt`. Ein Beispiel dafür könnte wie folgt aussehen:

```
- name: Prompts
  hosts: ansible_nodes
  vars_prompt:
    - name: user_input
      prompt: "Please enter the input of the user"
      private: no
  tasks:
    - name: Display the value
      debug:
        msg: "{{ user_input }}"
```

In diesem Fall wird der Benutzer aufgefordert, eine Eingabe zu tätigen. Die Aufforderung wird durch die Eigenschaft `prompt` spezifiziert. Die Eingabe selbst wird in der variable `user_input` abgelegt. Mit Hilfe der Eigenschaft `private` wird geregelt, ob die Eingabe zum Zeitpunkt der Erfassung sichtbar ist, oder nicht.

Facts

In Ansible gibt es die Möglichkeit, dass Variablen auch automatisch gesetzt werden. Diese werden als Facts bezeichnet. Facts sind Informationen über die Hosts, die in einem Playbook verwendet werden können. Diese Informationen können zum Beispiel die Adressen, der Hostname oder das Betriebssystem sein. Facts werden in der Regel automatisch von Ansible gesammelt, wenn ein Playbook ausgeführt wird. Sie können aber auch manuell gesetzt werden. Die Ausgabe von Facts kann dabei entweder direkt mit dem `ansible` Befehl und entsprechenden Parametern oder auch innerhalb eines Playbooks mit `ansible-playbook` erfolgen:

```
ansible -m setup -a 'filter=ansible_hostname' my_node1 -i hosts.yaml
```

TERMINAL

Die Ausgabe zeigt im Anschluss die gesammelten Facts an. Die meisten Facts selber stammen dabei immer aus dem `setup` Module. Häufig benutzte Facts sind:

- `ansible_hostname` : Hostname des Systems.
- `ansible_distribution` : Distribution des Systems.
- `ansible_distribution_version` : Version der Distribution.
- `ansible_processor` : Prozessor des Systems.
- `ansible_processor_cores` : Anzahl der Kerne des Prozessors.
- `ansible_processor_vcpus` : Anzahl der logischen Prozessoren.

Möchte man Facts innerhalb eines Playbooks nutzen, so kann dies mit Hilfe von Variablen durchgeführt werden:

```
- name: Ausgabe von Facts
  hosts: all
  tasks:
    - name: Zeige Hostname an
      debug:
        msg: "Hostname ist {{ ansible_hostname }}"
    - name: Zeige Distribution an
      debug:
        msg: "Distribution ist {{ ansible_distribution }}"
    - name: Zeige Version an
      debug:
        msg: "Version ist {{ ansible_distribution_version }}"
    - name: Zeige Prozessor an
      debug:
        msg: "Prozessor ist {{ ansible_processor }}"
    - name: Zeige Kerne an
      debug:
        msg: "Anzahl der Kerne ist {{ ansible_processor_cores }}"
    - name: Zeige logische Prozessoren an
      debug:
        msg: "Anzahl der logischen Prozessoren ist {{ ansible_processor_vcpus }}"
```

YAML

Custom Facts

Es gibt natürlich auch den Fall, dass man Informationen eines Managed Nodes benötigt, die nicht Bestandteil des `setup` Moduls sind. Hierbei spricht man von den **Custom Facts**. Wesentlich dabei ist:

- Custom Facts können in jeder beliebigen Sprache geschrieben werden.
- Custom Facts müssen am Ende `JSON` oder `.ini` Format haben.
- Custom Facts müssen im Verzeichnis `/etc/ansible/facts.d/` liegen.

Ein Beispiel für Custom Facts

Zunächst werden zwei neue, ausführbare Dateien angelegt. Die Datei `getdate.fac` ist im `JSON`, wohingegen die Datei `getdate2.fac` im `.ini` Format ist.

`getdate.fac` :

```
#!/bin/bash

echo {"\"date\"": \"`date`\"}"}
```

TERMINAL

`getdate2.fac` :

```
#!/bin/bash

echo [date]
echo date=`date`
```

TERMINAL

Die Dateien müssen im Anschluss ausführbar gemacht werden und in das Verzeichnis `/etc/ansible/facts.d/` kopiert werden:

```
chmod 700 getdate.fac
chmod 700 getdate2.fac
cp getdate.fac /etc/ansible/facts.d/
cp getdate2.fac /etc/ansible/facts.d/
```

TERMINAL

NOTE

Sollte das Verzeichnis `/etc/ansible/facts.d/` nicht existieren, so kann es mit dem Befehl `mkdir -p /etc/ansible/facts.d/` anlegen.

Verwendung der Custom Facts

Die Custom Facts sind aktuell lediglich auf dem Control Node vorhanden. Dieser ist nicht Bestandteil unserer Managed Nodes. Folglich müssen die Custom Facts auf die Managed Nodes übertragen werden. Um das zu erreichen, kann wieder ein Playbook verwendet werden:

```
- name: Create Custom Facts
  hosts: my_nodes
  user: root
  tasks:
    - name: Create the directory
      file:
        path: /etc/ansible/facts.d
        recurse: yes
        state: directory
    - name: Copy the fact 1
      copy:
        src: ./getdate.fac
        dest: /etc/ansible/facts.d/getdate.fac
        mode: 0755
    - name: Copy the fact 2
      copy:
        src: ./getdate2.fac
        dest: /etc/ansible/facts.d/getdate2.fac
        mode: 0755
    - name: Execute setup
      setup:
```

YAML

Jinja2

Jinja2 ist eine Engine für die Erstellung von Templates. Innerhalb von Ansible wird die Engine genutzt, damit dynamische Ausdrücke bzw. Muster erstellt und auch auf entsprechende Variablen und Facts von Managed Nodes zugegriffen werden kann. So ist man zum Beispiel in der Lage, dass Muster für Konfigurationsdateien erzeugt werden, die auf mehrere Managed Nodes im Anschluss deployed werden können. Es können folglich IP-Adressen, Hostnamen, Versionen und vieles mehr in die Templates eingefügt werden. (vgl. [ANS8])

Wichtig ist dabei, dass der komplette Prozess des Templating auf dem Control Node durchgeführt wird. Dadurch wird die Anzahl der benötigten Pakete auf dem Managed Node minimiert. Zudem wird die Menge an Daten, die Ansible an den Managed Node überträgt, reduziert. Ansible parst die Templates auf dem Control Node und überträgt nur die Informationen, die für den jeweiligen Task benötigt werden, an den Managed Node. Dadurch wird vermieden, dass alle Daten auf dem Control Node verarbeitet und anschließend an den Managed Node übertragen werden müssen. (vgl. [ANS8])

Ein Beispiel für die Verwendung von Jinja2

Im nachfolgenden Beispiel sollen einige Informationen des entsprechenden Managed Nodes wie z. B. der Hostname über Variablen in ein Template geschrieben werden. Das Template selbst trägt den Namen `test.j2`. Hier zeigt sich auch die relevante Dateierweiterung der Templates: `*.j2`. Templates sollen dabei immer innerhalb eines eigenen Ordners mit dem Namen `templates` abgelegt werden. In der Datei `test.j2` ist nachfolgender Inhalt zu finden:

```
# This is a test template
The hostname is {{ ansible_hostname }}
The IP address is {{ ansible_default_ipv4.address }}
The OS is {{ ansible_distribution }} {{ ansible_distribution_version }}
```

JINJA2

Im Anschluss benötigt man nun ein Playbook, welches auf das `template` Module zugreift:

```
- name: Write hostname
  hosts: all
  tasks:
    - name: Write hostname using jinja2
      template:
        src: templates/test.j2
        dest: /tmp/hostname
```

YAML

Eine wichtige Schreibweise fällt hier gleich vorweg auf. Variablen werden immer in doppelten geschweiften Klammern `{{ }}` geschrieben. Diese Schreibweise ist in Jinja2 Standard. Die Variablen, die hier verwendet werden, sind allesamt Facts, die durch das `setup` Modul bereitgestellt werden. Diese Facts sind Bestandteil des Verzeichnisses `/etc/ansible/facts.d/`. Hier greifen somit auch wieder die Custom Facts! (vgl. [ANS8])

Jinja2 Cheatsheet

Da es sich bei Jinja2 um eine komplexe Engine für Templates handelt, gibt es natürlich auch die unterschiedlichsten Kontrollstrukturen etc. In diesem Teilkapitel wird ein genereller Überblick über die wichtigste Syntax gegeben. Die Übersicht ist natürlich nicht vollständig! Je nach Anwendungszweck muss die Dokumentation eigenständig betrachtet werden!

Zugriff auf Variablen

```
Variable x has content: {{ x }}
```

JINJA2

Ausdrücke

```
Expression: {{ x + 1 }}
```

JINJA2

Schleife über mehrere Elemente

```
{% for x in <NameOfList> %}  
...  
{% endfor %}
```

JINJA2

Bedingte Anweisungen

```
{% if <Bedingung> %}  
...  
{% elif <Bedingung> %}  
...  
{% else %}  
...  
{% endif %}
```

JINJA2

Filter

```
{{ <Variable> | <Filter> }}
```

JINJA2

NOTE

In Jinja2 sind Filter mehr oder weniger auch eingebaute Funktionen, die für eine entsprechende Variable bzw. auch ein Array ausgeführt werden.

Funktionen / Makros

```
{% macro <Name> %}  
...  
{% endmacro %}
```

JINJA2

Kommentare

```
{# <Kommentar> #}
```

JINJA2

Ansible Beispiele

Innerhalb dieses Teilkapitels werden einige Aufgaben gestellt, die beschreiben sollen, was mit Ansible alles möglich ist!

Sicherheitskonzept für Ansible

Grundvoraussetzungen

Da Ansible immer nur auf einem einzigen Control Node installiert wird, ist es eine gute Idee sich ein eigenes Verzeichnis für Ansible zu erstellen. Innerhalb dieses Verzeichnisses werden alle Playbooks, Inventories und Variablen gespeichert. Das Verzeichnis trägt im Folgenden den Namen `ansible` :

```
mkdir ansible  
cd ansible
```

TERMINAL

SSH einrichten

Da Ansible für die Verbindung zu den Managed Nodes SSH verwendet, benötigt man einen entsprechenden Schlüssel. Dieser Schlüssel muss auf dem Control Node generiert werden (es muss kein Passphrase vergeben werden):

```
mkdir .ssh
cd .ssh
ssh-keygen -f ansible -t ecdsa -b 521
```

Da Ansible für sehr viele Aktionen allerdings auch einen administrativen Zugriff benötigt, ist es sinnvoll, einen weiteren Schlüssel nur für Rootzugriffe zu erstellen (für diesen Schlüssel sollte ein Passphrase vergeben werden):

```
ssh-keygen -f root -t ecdsa -b 521
```

Vorbereitende Schritte auf den Managed Nodes

Da auf allen Managed Nodes ein User `ansible` existieren soll, muss dieser natürlich noch erzeugt werden. Hierfür kann ein entsprechendes Skript benutzt werden, dass immer wieder ausgeführt werden kann. Das Skript könnte den Namen `prepare_managed_node.sh` haben und folgendermaßen aussehen:

```
#!/bin/bash
set -e
PUBKEY=""
id -u ansible > /dev/null 2>&1 || \
adduser ansible --disabled-password \
--gecos "" --quiet
mkdir -p /home/ansible/.ssh
echo "$PUBKEY" \
> /home/ansible/.ssh/authorized_keys
chown -R ansible /home/ansible/.ssh
apt update
apt install sudo python
grep -q ansible /etc/sudoers || \
echo "ansible ALL = (ALL) \
NOPASSWD: ALL" >> /etc/sudoers
```

Erläuterung zum Skript:

- `set -e` : Bei einem Fehler wird das Skript abgebrochen.
- `PUBKEY` : Hier muss der öffentliche Schlüssel des Control Nodes eingetragen werden. Dieser wird im Folgenden benötigt, damit der User `ansible` auf den Managed Node zugreifen kann.
- `id -u ansible > /dev/null 2>&1` : Überprüft, ob der User `ansible` existiert. Die Ausgabe wird unterdrückt.
- `adduser ansible --disabled-password --gecos "" --quiet` : Der User `ansible` wird angelegt. Der User hat kein Passwort und wird nicht nach einem Passwort gefragt.
- `mkdir -p /home/ansible/.ssh` : Das Verzeichnis `.ssh` wird für den User `ansible` erzeugt.
- `echo "$PUBKEY" > /home/ansible/.ssh/authorized_keys` : Der öffentliche Schlüssel des Control Nodes wird in die Datei `authorized_keys` geschrieben. Dadurch kann der User `ansible` auf den Managed Node zugreifen.
- `chown -R ansible /home/ansible/.ssh` : Hier wird das Verzeichnis `.ssh` dem User `ansible` zugewiesen.
- `apt update` : Das System wird aktualisiert.
- `apt install sudo` : `sudo` wird installiert (falls nicht vorhanden).
- `grep -q ansible /etc/sudoers || echo "ansible ALL = (ALL) NOPASSWD: ALL" >> /etc/sudoers` : Hier wird der User `ansible` in die Datei `/etc/sudoers` eingetragen. Dadurch hat der User `ansible` die Berechtigung, alle Befehle ohne Passwort auszuführen.

Das Skript muss im Anschluss ausführbar gemacht werden:

```
chmod 700 prepare_managed_node.sh
```

Es ist nun an der Zeit, dass das Skript auf den entsprechenden Managed Nodes ausgeführt wird. Der schnellste Weg hierfür ist mit Hilfe von SSH. Wichtig dabei ist, dass man auf dem Managed Node einen entsprechenden User hat:

```
scp prepare_managed_node.sh <User>@<Managed_Node>:/tmp/
```

Im Anschluss kann das Skript ebenfalls via SSH auf dem Managed Node bzw. direkt mit einem User auf den Rechner zugegriffen und das Skript direkt ausgeführt werden:

```
ssh <User>@<Managed_Node> "/tmp/prepare_managed_node.sh"
```

Vorbereitende Schritte auf dem Control Node

Auf dem Control Node wird eine entsprechende Inventory benötigt, welches sämtliche Managed Nodes beinhaltet und weiterhin auch eine Konfiguration für Ansible:

Inventory

Das Inventory hängt natürlich wieder ganz davon ab, welche Hosts man hat. Ein beispielhafter Aufbau kann wie folgt aussehen:

```
debian_hosts:
  hosts:
    deb1:
      ansible_host: <IP>

master:
  children:
    debian_hosts:
```

Konfiguration

In den bisherigen Konfigurationen wurde lediglich spezifiziert, was das Inventory ist, welches Ansible nutzen soll. Jetzt wird die Konfiguration um zwei weitere Einstellungen ergänzt:

- **remote_user :**
Mit diesem Parameter wird spezifiziert, welcher User auf den Managed Nodes genutzt wird, damit Befehl ausgeführt werden können. Falls dieser Parameter nicht angegeben ist, so wird immer der aktuell angemeldete Benutzer auf dem Control Node verwendet für die Ausführung von Ansible.
- **private_key_file :**
Hierrüber wird der Pfad zur privaten Schlüsseldatei angegeben, die für die Authentifizierung via SSH genutzt wird. Falls der Parameter nicht angegeben sein sollte, so wird die Schlüsseldatei des aktuell angemeldeten Benutzers verwendet!

Es ergibt sich somit folgende Konfiguration:

```
Unresolved directive in 5.6.1_Sicherheitskonzept_fuer_Ansible.adoc -
include:../../Skripte/Ansible/ansible.cfg[]
```

Konfiguration für SSH anpassen

Im Folgenden wird ein neues Playbook mit dem Namen `configure_ssh_play.yaml` angelegt. Ziel des Playbooks ist es, dass die Konfiguration für SSH angepasst wird. Zu den Aufgaben des Playbooks gehören:

- Der Daemon muss so konfiguriert werden, dass er nur noch eine Authentifizierung mit Schlüsseln erlaubt ist. Folglich ist die Anmeldung mit einem Passwort deaktiviert.
 - Um die Änderung sofort wirksam zu machen, wird der Daemon neu gestartet.
- In einem zweiten Schritt wird der angelegte Schlüssel für den User `root` in die Datei `.authorized_keys` geschrieben. Dadurch kann der User `root` sich mit dem Schlüssel anmelden.

```
nano configure_ssh_play.yaml
```

TERMINAL

```
Unresolved directive in 5.6.2_SSH_Konfiguration_anpassen.adoc -
include:../../../../Skripte/Ansible/configure_ssh_play.yaml[]
```

YAML

Erläuterungen zum Playbook:

- `name: Configure SSH and deploy root key` : Der Name des Playbooks.
- `hosts: school` : Das Playbook wird auf allen Managed Nodes ausgeführt.
- `become: yes` : Das Playbook wird mit den Rechten von Root ausgeführt.
- `tasks:` : Hier werden die Aufgaben des Playbooks definiert.

Das Playbook selbst besteht aus zwei Aufgaben:

Aufgabe 1: SSH Konfiguration anpassen

- `name: Disable password authentication` : Der Name der Aufgabe.
- `lineinfile:` : Hierbei handelt es sich um ein Modul in Ansible, wodurch Zeilen in Dateien geändert oder hinzugefügt werden können. Es kann aber auch eine Prüfung stattfinden, ob eine Zeile überhaupt existiert. Zu den Parametern des Moduls gehören dabei:
 - `path` : Der Pfad zur Datei, die bearbeitet werden soll.
 - `regexp` : Hier wird ein regulärer Ausdruck angegeben, der in der Datei gesucht werden soll. Der Ausdruck `^PasswordAuthentication` sucht nach der Zeile, die mit dem String `PasswordAuthentication` beginnt.
 - `line` : Hier wird die Zeile angegeben, die in die Datei geschrieben werden soll.
 - `state` : Stellt sicher, dass die Zeile vorhanden sein muss.
 - `backup` : Mit dieser Option kann spezifiziert werden, ob eine Sicherungskopie vor der Änderung angelegt werden soll, oder nicht.
- `notify` : Hier wird angegeben, dass eine Benachrichtigung an den Handler `Restart SSH` gesendet werden soll, wenn die Aufgabe erfolgreich war.

Aufgabe 2: Schlüssel für den User `root` bereitstellen

- `name: Deploy key` : Der Name der Aufgabe.
- `authorized_key` : Hierbei handelt es sich um ein Modul in Ansible, wodurch Schlüssel in die Datei `authorized_keys` geschrieben werden können. Zu den Parametern des Moduls gehören dabei:
 - `user` : Der User, für den der Schlüssel bereitgestellt werden soll.

- **key** : Diese Eigenschaft gibt den Pfad bzw. den Schlüssel an, der in die Datei `authorized_keys` geschrieben werden soll. Um den Schlüssel zu erhalten wird das Plugin `lookup` verwendet. Dieses Modul erlaubt es, dass Dateien gelesen werden können. Hierzu muss lediglich der Pfad zur Datei angegeben werden.

Handler

- **name** : Restart SSH : Der Name des Handlers.
- **service** : Hierbei handelt es sich um ein Modul in Ansible, wodurch Dienste gestartet, gestoppt oder neu gestartet werden können. Zu den Parametern des Moduls gehören dabei:
 - **name** : Der Name des Dienstes, der gestartet, gestoppt oder neu gestartet werden soll.
 - **state** : Hier wird der Zustand des Dienstes angegeben. In diesem Fall soll der Dienst neu gestartet werden.

Hostname setzen

Da teilweise Managed Nodes nicht immer die passenden Hostnamen haben, ist es sinnvoll, diese mit Ansible zu setzen. Hierzu wird ein Playbook mit dem Namen `set_hostname_play.yaml` angelegt. Ziel des Playbooks ist es, den Hostnamen der Managed Nodes zu setzen. Das Playbook selbst hat folgenden Aufbau:

```
Unresolved directive in 5.6.3_Hostname_setzen.adoc -
include:../../../../Skripte/Ansible/set_hostname_play.yaml[]
```

YAML

Erläuterungen zum Playbook:

- **name** : Set hostname of node : Der Name des Playbooks.
- **hosts** : all : Das Playbook wird auf allen Managed Nodes ausgeführt.
- **become** : yes : Das Playbook wird mit den Rechten von Root ausgeführt.
- **tasks** : : Hier werden die Aufgaben des Playbooks definiert. Es gibt zwei Aufgaben:
 - **name** : Adjust /etc/hosts : Der Name der Aufgabe.
 - **replace** : Hierbei handelt es sich um ein Modul in Ansible, wodurch Zeilen in Dateien geändert oder hinzugefügt werden können. Es kann aber auch eine Prüfung stattfinden, ob eine Zeile überhaupt existiert. Zu den Parametern des Moduls gehören dabei:
 - **path** : Der Pfad zur Datei, die bearbeitet werden soll.
 - **regex** : Es wird ein regulärer Ausdruck erstellt, nachdem innerhalb des Files gesucht werden soll. Im Beispiel ist der Ausdruck `(\s*){{ ansible_hostname }}(\s*) . (\s*)` ist ein regulärer Ausdruck, der alle Leerzeichen vor und nach dem Hostnamen sucht. Die Variable `{{ ansible_hostname }}` wird durch den Hostnamen des Managed Nodes ersetzt, der gerade eben im Zugriff ist.
 - **replace** : Hier wird die Zeile angegeben, die in die Datei geschrieben werden soll. Die Variable `{{ inventory_hostname }}` ist der Hostname, der im Inventory angegeben ist.
 - **name** : Set hostname : Der Name der zweiten Aufgabe.
 - **hostname** : Hierbei handelt es sich um ein Modul in Ansible, wodurch der Hostname des Managed Nodes gesetzt werden kann. Zu den Parametern des Moduls gehören dabei:
 - **name** : Hier wird der Hostname angegeben, der gesetzt werden soll. Die Variable `{{ inventory_hostname }}` ist der Hostname, der im Inventory angegeben ist.

Systemaktualisierungen

Unattended Upgrades

Hat man mehrere Systeme, so müssen diese natürlich auch ständig aktualisiert werden. Nicht nur wegen Sicherheitslücken, sondern auch um neue Funktionen erhalten zu können. Ansible bietet natürlich auch hierfür Möglichkeiten, dass Aktualisierungen auf den Managed Nodes automatisch durchgeführt werden können. Für diesen Zweck wird ein Playbook mit dem Namen `apt_auto_upgrade.yaml` angelegt. Diese Playbook soll sich darum kümmern, dass die sogenannten **Unattended Upgrades** durchgeführt werden können.

NOTE

Unattended Upgrades sind ein Mechanismus unter Linux, wodurch Sicherheitsupdates und andere Paketaktualisierungen automatisch im Hintergrund ausgeführt werden können. Der Benutzer muss somit nicht mehr eingreifen und das System bleibt generell immer aktuell.

Damit die Unattended Upgrades funktionieren, müssen diese zunächst aktiviert werden. Dies geschieht über die Datei `/etc/apt/apt.conf.d/20auto-upgrades`. Diese Datei muss mit dem folgenden Inhalt angelegt werden:

```
APT::Periodic::Update-Package-Lists "1";
APT::Periodic::Unattended-Upgrade "1";
```

TERMINAL

Neben der Aktivierung ist es auch wichtig, dass die Unattended Upgrades konfiguriert werden. Dies wird über die Datei `/etc/apt/apt.conf.d/50unattended-upgrades` durchgeführt. Innerhalb dieser Konfiguration wird spezifiziert, welche Pakete auf dem System automatisch aktualisiert werden dürfen:

```
Unattended-Upgrade::Allowed-Origins {
    "${distro_id}:${distro_codename}";
    "${distro_id}:${distro_codename}-security";
    "${distro_id}:${distro_codename}-updates";
};
```

TERMINAL

Diese Dateien können mit Hilfe des `template` Moduls in Ansible angelegt werden. Folglich benötigt man einen Ordner mit dem Namen `templates`, in dem die beiden Dateien liegen. Die Dateien haben den Namen `20auto-upgrades.j2` und `50unattended-upgrades.j2`.

Im Anschluss kann das Playbook `configure_unattended_upgrades_play.yaml` angelegt werden. Dieses Playbook hat den folgenden Inhalt:

```
Unresolved directive in 5.6.4_Systemaktualisierungen.adoc -
include:../../../../Skripte/Ansible/configure_unattended_upgrades_play.yaml[]
```

YAML

Erläuterungen zum Playbook:

- `name: Configure unattended upgrades` : Der Name des Playbooks.
- `hosts: all` : Das Playbook wird auf den Managed Nodes ausgeführt, die in der Gruppe `private` sind.
- `become: yes` : Das Playbook wird mit den Rechten von Root ausgeführt.
- `tasks:` : Hier werden die Aufgaben des Playbooks definiert. Es gibt drei Aufgaben:
 - `name: Install unattended upgrades package` : Der Name der Aufgabe.
 - `apt` : Hierbei handelt es sich um ein Modul in Ansible, wodurch Pakete installiert werden können. Zu den Parametern des Moduls gehören dabei:
 - `pkg` : Der Name des Pakets, welches installiert werden soll. In diesem Fall ist es `unattended-upgrades`.
 - `state` : Der Zustand in dem sich das Paket befinden soll. In diesem Fall soll das Paket installiert werden bzw. installiert sein.

- `update_cache` : Mit diesem Parameter soll der Cache aktualisiert werden.
- `name: Copy the template 20auto-upgrades` : Der Name der Aufgabe.
- `template` : Hierbei handelt es sich um ein Modul in Ansible, wodurch Templates kopiert werden können. Zu den Parametern des Moduls gehören dabei:
 - `src` : Der Pfad zur Quelldatei. In diesem Fall ist es die Datei `20auto-upgrades.j2`.
 - `dest` : Der Pfad, wohingegen die Datei kopiert werden soll. In diesem Fall ist es die Datei `/etc/apt/apt.conf.d/20auto-upgrades`.
 - `owner` : Mit diesem Parameter wird der Eigentümer der Datei spezifiziert. In diesem Fall ist es `root`.
 - `group` : Hiermit kann die Gruppe der Datei angegeben werden. In diesem Fall ist es `root`.
 - `mode` : `mode` spezifiziert die Zugriffsrechte der Datei. Hier liegen die Rechte bei `0644`.

Die zweite Aufgabe ist fast identisch mit der ersten Aufgabe. Der einzige Unterschied ist, dass die Quelldatei `50unattended-upgrades.j2` und die Zieldatei `/etc/apt/apt.conf.d/50unattended-upgrades` ist.

apt dist-upgrade

Der `apt dist-upgrade` ist ein Befehl, der verwendet wird, um Pakete zu aktualisieren und gleichzeitig Abhängigkeiten zu berücksichtigen. Es ist eine erweiterte Version des `apt upgrade` Befehls. Der Hauptunterschied zwischen den beiden besteht darin, dass `apt dist-upgrade` auch in der Lage ist, Pakete zu entfernen oder neue Pakete zu installieren, um die Abhängigkeiten der aktualisierten Pakete zu erfüllen.

Für diesen Zweck kann ein Playbook mit dem Namen `apt_dist_upgrade_play.yaml` angelegt werden. Das Playbook hat den folgenden Inhalt:

```
Unresolved directive in 5.6.4_Systemaktualisierungen.adoc -
include:../../../../Skripte/Ansible/apt_dist_upgrade_play.yaml[]
```

YAML

Erläuterungen zum Playbook:

- `name: Make an apt dist-upgrade` : Der Name des Playbooks.
- `hosts: all` : Das Playbook wird auf den Managed Nodes ausgeführt, die in der Gruppe `school` sind.
- `become: yes` : Das Playbook wird mit den Rechten von Root ausgeführt.
- `tasks:` : Hier werden die Aufgaben des Playbooks definiert. Es gibt drei Aufgaben:
 - `name: Perform the upgrade` : Der Name der ersten Aufgabe.
 - `apt` : Hierbei handelt es sich um ein Modul in Ansible, wodurch Pakete aktualisiert und auch installiert werden können. Zu den Parametern des Moduls gehören dabei:
 - `upgrade` : Der Upgrade-Typ. In diesem Fall ist es `dist`, was bedeutet, dass alle Pakete aktualisiert werden sollen.
 - `update_cache` : Mit diesem Parameter soll der Cache aktualisiert werden.
 - `name: Check if a reboot is required` : Der Name der zweiten Aufgabe.
 - `stat` : `stat` ist ein Modul in Ansible wodurch der Status einer Datei überprüft werden kann. Parameter hierfür sind:
 - `path` : Der Pfad zur Datei, die überprüft werden soll. In diesem Fall ist es die Datei `/var/run/reboot-required`.

- `get_checksum` : Mit diesem Parameter kann angegeben werden, ob die Prüfziffer der Datei überprüft werden soll. Hier wird die Berechnung nicht durchgeführt, da `no` .
- `register: reboot_required_file` : `register` ist in der Lage, dass das Ergebnis der Prüfung in einer Variable abgelegt wird. Die Variable heißt hier `reboot_required_file` .
- `name: Make the reboot (if required)` : Der Name der dritten Aufgabe.
- `reboot` : Mit dem Modul `reboot` kann ein Neustart des Systems durchgeführt werden.
- `when: reboot_required_file.stat.exists == true` : Der Parameter `when` ist in der Lage auf entsprechende Bedingungen zu prüfen und somit Aufgaben nur auszuführen, falls die Bedingung erfüllt ist. In diesem Fall wird überprüft, ob die Datei `/var/run/reboot-required` existiert. Wenn dies der Fall ist, wird das System neu gestartet.

Docker installieren

Ansible kann natürlich auch verwendet werden, damit die unterschiedlichsten Pakete automatisiert auf mehreren Systemen installiert werden. Das nachfolgende Playbook `install_docker_play.yaml` zeigt, wie man mit Hilfe von Ansible Docker auf mehreren Managed Nodes gleichzeitig installieren kann!

```
Unresolved directive in 5.6.5_Docker_installieren.adoc -
include:../../../../Skripte/Ansible/install_docker_play.yaml[]
```

YAML

Erläuterungen zum Playbook:

- `name: Install docker (Debian)` : Der Name des Playbooks.
- `hosts: school` : Das Playbook wird auf den Managed Nodes ausgeführt, die in der Gruppe `school` sind.
- `become: yes` : Das Playbook wird mit den Rechten von Root ausgeführt.

Aufgabe 1: Installation of necessary packages

Über das Modul `apt` werden die Pakete `git`, `ca-certificates`, `curl` und `lsb-release` installiert. Diese Pakete sind notwendig, um Docker zu installieren. Da die Pakete installiert werden sollen (= `state: present`), wird im Voraus der Cache aktualisiert (= `update_cache: yes` und entspricht einem `apt update`).

Aufgabe 2: Create directory for keyrings

Über das Modul `file` wird das Verzeichnis `/etc/apt/keyrings` erstellt. Das Verzeichnis wird mit den Rechten `0755` angelegt. Das Verzeichnis wird dabei nur angelegt, falls es noch nicht vorhanden sein sollte.

Aufgabe 3: Download docker key

Mit Hilfe des Moduls `get_url` wird der Schlüssel für Docker heruntergeladen. Der Schlüssel wird im Anschluss in das Verzeichnis `/etc/apt/keyrings` in die Datei `docker.asc` gespeichert. Die Datei wird mit den Rechten `0644` angelegt.

Aufgabe 4: Find the right architecture

Das Modul `set_fact` wird verwendet, um eine neue Variable zu setzen. In diesem speziellen Fall wird versucht, dass die Architektur des Systems herausgefunden werden kann. Es wird geprüft, ob die Architektur `x86_64` ist. Wenn dies der Fall ist, wird die Variable `arch` auf `amd64` gesetzt. Andernfalls wird die Variable auf den Wert von `ansible_architecture` gesetzt.

Aufgabe 5: Add repository for docker

Über das Modul `copy` wird das Repository für Docker in die Datei `/etc/apt/sources.list.d/docker.list` geschrieben. Dabei wird die Variable `arch` verwendet, um die Architektur des Systems anzugeben.

Aufgabe 6: `apt update (after repository changes)`

Über das Modul `apt` wird der Cache aktualisiert. Dies ist notwendig, da das Repository für Docker hinzugefügt wurde. Das Update wird mit dem Parameter `update_cache: yes` durchgeführt.

Aufgabe 7: `Install docker packages`

Mit Hilfe des Moduls `apt` werden die notwendigen Pakete für Docker installiert und auch hier wieder der Cache aktualisiert.

Weitere Playbooks

Kerberos installieren und konfigurieren

Das Playbook stellt beispielhaft die Konfiguration von Kerberos gegen die Domäne `b11b.local` dar:

```
Unresolved directive in 5.6.6>Weitere_Playbooks.adoc -  
include:../../../../Skripte/Ansible/configure_kerberos_play.yaml[]
```

YAML

Die entsprechende Konfiguration der Datei `krb5.conf` muss entsprechend den notwendigen Parametern angepasst werden!